PulseQoS: Design Document v2

Tata Elxsi TELIPORT Season 4 · Case Study 3: Adaptive QoS Engine for Mixed Home Broadband Traffic
Team: Manas Girdhar (Team Leader), Meenakshi Satpathy · Birla Institute of Technology, Mesra · v2, 27 Oct 2026

> **Tagline:** the call never notices the download.
> **One-line pitch:** PulseQoS is an open Linux QoS controller that recognises traffic without reading it, follows the real link speed within a second, accepts a temporary user intent, and **predicts the effect of every policy change before users feel it**, using a learned *Experience Twin* of the household link. If a change still hurts, it rolls back.

1. Summary

PulseQoS runs on a home router. It protects interactive traffic (calls, gaming) when the link is busy or its capacity drops, while making sure bulk traffic still moves. It is built on proven Linux mechanisms (CAKE, HTB, IFB, conntrack) and adds a control layer on top of them, in two parts.

Deterministic core (always on, always the baseline):

1. Two-speed control: a fast loop that tracks link capacity, and a slow loop for policy changes behind a safety gate.
2. Focus Mode intents with expiry, a priority ceiling and a daily Fairness Budget.
3. Validate → bounded live trial → keep or roll back, plus a watchdog that restores a known safe state.
4. One-command reproducible experiments against strong existing systems.

The AI layer that makes PulseQoS different. Each part must beat a deterministic twin, or it is switched off.

AI 1 Conformal traffic classifier
Boosts a flow only when the prediction set is a single interactive class. Wrong boosts are then bounded by a chosen rate (e.g. 1%) *by construction* on traffic like the calibration data.

AI 2 Experience Twin
A learned model of the household link that predicts the next 10 s of call delay for each candidate action. Unsafe policies are rejected *before* users feel them.

AI 3 Zero-click Focus
Detects that a call has started on a loaded link and protects it automatically, with Fairness Budget accounting and one-tap undo.

What we deliberately do not do: no chatbot and no LLM in the control path. The case study asks that AI *"improves a measurable outcome rather than merely adding a conversational interface"*. Every AI part above has a number it must improve.

2. Why this wins: what others will and won't do

Typical submission	PulseQoS
"ML classifies traffic, then prioritise it"	Builds on CAKE and is benchmarked *against* CAKE, CAKE's own autorate and cake-autorate, not just against "no QoS"

Classifier accuracy (F1) only	A statistical bound on the harmful error (wrong boost), plus time-to-classify
AI changes the network live and hopes	The Experience Twin screens every change first; the safety gate verifies it live; the watchdog restores a safe state
A user toggle for "gaming mode"	Temporary intents with expiry and Fairness Budget; zero-click protection with undo
A pretty dashboard	A real measured pilot already (section 10), raw data committed, and every chart reproducible from one command

3. Problem and hard constraints

A home connection carries calls, gaming, voice, adaptive video, downloads, updates and backups at the same time. Large queues, not low bandwidth, are what ruin interactive traffic. The case study's scenarios: (1) an ISO download during a video conference; (2) WAN drops from 100 to 20 Mbps; (3) three TVs plus a latency-sensitive game.

Constraint (case study)	How PulseQoS meets it
Do not decrypt application traffic	Classification uses flow metadata only (sizes, timing, direction, protocol). No payload, no SNI or domain logging.
Linux tc or another open, inspectable mechanism	CAKE (primary), HTB + fq_codel (hard-floor variant), IFB, tc actions `ctinfo`, `pedit`, `mirred`
IPv4 and, where available, IPv6	Both stacks in every scenario; parity reported
Policy rollback; no starvation of low priority	Versioned policy bundles, last-known-good rollback; HTB floors (hard) or CAKE minimum-progress monitoring (soft)
Classifiers expose confidence; allow correction	Conformal prediction sets plus calibrated scores; user correction in UI and API
Document third-party licences	Licence table in repo, including datasets
Generated data and impairment settings included	All netem settings, traffic scripts, seeds and raw results committed
Emulated hardware labelled as emulation	Every chart is labelled "Linux namespace emulation"
No credentials in source control	Config via environment; example secrets are fictitious; secret scanning in CI
Remediation bounded, observable, reversible; failure returns to a safe state	Fixed action menu, Experience Twin pre-check, validator, bounded trial, fast-abort, watchdog

Expected-output checklist (case study) → where it is delivered

Expected output	Delivered as
Traffic-class and link-capacity estimator	Modules 1 and 2
Dynamic QoS policy engine and enforcement module	Modules 4, 6 and section 6
Dashboard: latency, jitter, loss, throughput, queue depth, fairness	Module 7
Automated baseline-vs-optimised experiments	Section 9, `make bench`

API or UI for temporary service intent	Module 5 (REST + dashboard)
Architecture diagram: device, edge, network, data, analytics, UI	Section 4 (six-layer view)
Setup guide with exact versions, commands, verification	`SETUP.md`: kernel, iproute2, Python and package versions pinned; `make verify`
Scripted demo: reset, inject condition, collect evidence, report	`make demo SCENARIO=S2`
Known-limitations section	Section 13

4. Architecture: six layers, one closed loop, two speeds

Layer	Components
UI	Focus Mode app and REST API (LAN-only, authenticated); dashboard with a "Why" panel and audit timeline
Analytics & control	**Fast loop (≤ 1 s):** capacity estimator → shaper rate. **Slow loop (10–30 s):** policy controller → AI Experience Twin pre-check → safety gate. Watchdog.
Data	Flow table (conntrack + features), telemetry time series, versioned policy bundles, audit log, Twin training set
Edge (router)	AI Conformal flow classifier → class written to the flow's conntrack mark → tc enforcement (CAKE on WAN egress, CAKE on IFB for download)
Network	WAN bottleneck (emulated: token bucket + netem), reflectors for delay probes
Device	Home devices (emulated): laptop call, console, three TVs, large download; IPv4 and IPv6

> FIX **Why two speeds.** In v1, every change went through a 10–30 s trial and a minimum hold time, and so did shaper-rate cuts. During a 100 → 20 Mbps drop that would delay the one action that prevents a queue. In v2, **rate decreases from the capacity estimator act immediately** and are exempt from hold time and trial; rate increases are slow and bounded. Only tin mappings, intents and class settings go through the safety gate. If the fast loop changes the rate during a trial, the trial is marked *inconclusive* and repeated, so a capacity change is never blamed on a policy.

5. Module specifications

Modules communicate with small JSON messages on a local bus, so any module can be replaced by a stub in tests. Each module lists its input, output, state, failure handling and success condition, as the case study requires.

Module 1: Conformal flow classifier AI 1

- **Input:** per-flow metadata from conntrack and packet sampling: packet-size statistics, inter-arrival times, direction ratio, burst pattern, protocol (TCP/UDP/QUIC), port range, flow age. No payload bytes.
- **Classes:** Call-Audio, Call-Video, Gaming, Streaming, Interactive-Web, Bulk, Unknown. FIX v1 had one "Interactive-RT" class; audio and video of a call are split because they belong in different CAKE tins (section 6).
- **Model:** LightGBM on early-flow and rolling-window features.

- **Confidence = conformal prediction:** on a held-out calibration set, compute nonconformity scores and choose a threshold so that the prediction set contains the true class with probability ≥ 1 − α. We use the class-conditional (Mondrian) variant, so this holds per class. **A flow is boosted only if its prediction set is a single interactive class.** If the true class is not interactive, that singleton set has missed the true class, so **P(wrong boost) ≤ α** (marginal, under exchangeability with the calibration data). Any other set means Unknown, which means best effort.
- **Honest limit:** the guarantee holds for traffic like the calibration data. A drift monitor tracks the empirical miss rate on user-confirmed flows; if it exceeds α, boosting falls back to the deterministic rules until recalibration.
- **Deterministic twin (baseline):** port and hostname rules (qosify-style).
- **State:** flow table with class, prediction set, user override. Overrides win immediately for enforcement; they enter training data only after validation.
- **Failure:** model crash or timeout → all flows best effort; nothing is boosted on a guess.
- **Success:** macro-F1 and confusion matrix; **wrong-boost rate vs the target α**; abstention rate; **time-to-classify** NEW ; results on held-out unseen apps, VPN and QUIC.
- **Data** FIX : train and test on **real application captures** recorded by the team with consent (call, game, streaming, download), plus public labelled datasets where the licence allows (e.g. ISCX VPN-nonVPN 2016, CESNET-QUIC22). Synthetic generators are used for load only, never as classifier ground truth. Splits are by capture session.

Module 2: Capacity estimator (fast loop)

- **Input:** delay probes to several reflectors (1/s, faster under load); achieved rate per direction from `tc -s`.
- **Method** FIX : **one-way delay** per direction using ICMP timestamp probes (as `tsping` does for cake-autorate) or `irtt`. Clocks need not be synchronised, because only the *increase* over a per-direction baseline is used. When OWD is unavailable (some ISPs block ICMP timestamps), fall back to RTT and cut only the direction whose achieved rate is near its shaper limit.
- **Control law:** AIMD. If loaded delay rises above baseline by a threshold, cut that direction's rate multiplicatively at once. If the link is loaded and delay stays low, probe upward slowly. Baseline tracked by a slow EWMA.
- **Failure:** reflectors silent → freeze at the last safe rate, lower confidence, never raise without fresh evidence.
- **Success:** estimation error against the netem rate; time to settle after 100 → 20 Mbps; size of the delay spike; per-direction attribution accuracy.

Module 3: Experience Twin AI 2 NEW

- **What:** a learned model that predicts the **next-10-second p95 delay of protected flows** (and bulk goodput) given the current state and a candidate action.
- **Features:** per-tin backlog and drops, per-class demand, capacity estimate and trend, active intents, candidate action.
- **Model:** LightGBM quantile regression (P50 and P90), small enough for router CPU.
- **Training data:** collected **offline in the lab** by running randomised safe actions across all scenarios. No exploration happens on real users.
- **Use:** before the safety gate, the controller scores every action in the safe menu. Actions whose predicted P90 call delay exceeds the target, or whose predicted bulk goodput falls below its floor, are dropped. The best remaining action goes to the live trial.
- **Deterministic twin (baseline):** the same controller and safety gate without pre-screening.

- **Success:** fewer harmful live trials and fewer **harm-seconds** (seconds a protected flow spends above its delay target) at equal or better p95 delay. Plus prediction error (pinball loss) on held-out runs.
- **Honest limit:** trained in emulation, so there is a sim-to-real gap. The live trial and fast-abort still guard every change, and the Twin can only *remove* actions, never bypass the gate.

Module 4: Policy controller (slow loop)

- **Output:** a candidate policy bundle: flow-class → tin mapping, intent effects, HTB rate/ceil (variant), shaper margin.
- **Method:** deterministic rules plus a small constrained optimiser over the fixed safe-action menu. It minimises $C = \alpha \cdot L95 + \beta \cdot J + \gamma \cdot P + \lambda \cdot F + \mu \cdot S$, where each term is normalised by a fixed reference scale (e.g. 50 ms delay, 10 ms jitter, 1% loss, one switch per minute). L95, J and P are the p95 delay, jitter and loss of protected flows; F is a demand-aware shortfall below min(demand, floor); S counts switches.
- **Stability:** hysteresis, a minimum hold time and a maximum number of changes per minute (slow loop only).
- **Optional tuner (stretch):** a contextual bandit (LinUCB) choosing within the Twin-screened, validator-approved menu. It is kept only if it wins in A/B runs.
- **Failure:** any exception → last known-good bundle.

Module 5: Intent manager (Focus Mode + Zero-click Focus) AI 3

- **Manual intent:** `POST /intents {device, class, duration}`. Every intent auto-expires; the boosted class has a ceiling (hard in HTB, monitored in CAKE). Each device has a daily Fairness Budget of boosted minutes. Conflicting intents: the earlier one keeps priority and the user is told.
- **Zero-click Focus** NEW : when the conformal classifier emits a singleton Call-Audio/Call-Video set for a new flow while the link is loaded, the system creates a short intent (e.g. 10 min, renewed while the call lasts). It charges the device's Fairness Budget and shows "Protected your call · Undo".
- **Baseline:** manual intent only.
- **Success metric: time from call start to protection**; harm-seconds in the first minute of a call.
- **Security** FIX : the API listens on the LAN only. Each user has a paired device token; requests are rate-limited; the owner can revoke; every intent is audited. A device cannot extend its own boost past its budget.
- **Identity:** device = MAC + DHCP lease + IPv6 neighbour entries (handles rotating IPv6 privacy addresses); flow = conntrack 5-tuple.
- **Precedence:** active intent > user correction > classifier > best effort.
- **Failure:** intent store lost → drop all intents → neutral policy.

Module 6: Safety gate

1. **Validate:** reject a candidate if floors exceed the shaped rate; any rate exceeds the capacity estimate; an intent ceiling or budget is exceeded; or the slow-loop hold time is not met.
2. **Apply and record** the whole bundle as one version. `tc -batch` is not atomic, so the controller reads the live state back after every apply. On any mismatch it rebuilds the last known-good bundle; if that also fails, the watchdog applies the neutral policy.
3. **Bounded live trial** (10–30 s) against the pre-change window. FIX The trial is marked *inconclusive* if the capacity estimate, the active intents or the set of active flows changes materially during the trial.
4. **Fast abort:** roll back immediately if protected p95 delay exceeds its pre-change value by a margin in two consecutive 1 s windows, or interactive loss jumps past a threshold. Margins are calibrated on baseline runs.

5. **Decide:** keep only if protected metrics improved and bulk stayed above its floor or minimum-progress target.
6. **Watchdog:** a separate process; on a missing heartbeat it applies plain CAKE at a conservative rate.

Streaming harm can be hidden by playback buffers for 10–30 s, so streaming is judged over a longer window using throughput sufficiency (is the stream receiving at least its observed bitrate).

Module 7: Telemetry, dashboard, audit, privacy

- **Charts:** latency (p50/p95/p99), jitter, loss, throughput, queue depth per tin, Jain's index, RPM. A "Why" panel for every decision shows the trigger, the Twin's prediction vs the measured outcome, the trial result, and keep or rollback.
- **Privacy** FIX : class-level metadata only; no payload, domains or SNI stored; short retention; per-device detail visible only to the account owner.

6. Linux implementation (corrected and checked against tc man pages)

Class → tin mapping (CAKE diffserv4)

From tc-cake(8), diffserv4 has four tins: **Bulk** (CS1, LE; 6.25% threshold), **Best Effort** (100%), **Video** (AF4x, AF3x, CS3, AF2x, CS2, TOS4, TOS1; 50%) and **Voice** (CS7, CS6, EF, VA, CS5, CS4; 25%).

PulseQoS class	DSCP written	CAKE tin	Why
Call-Audio, Gaming	EF (46)	Voice	Small packets, latency-critical; well within 25%
Call-Video FIX	AF41 (34)	Video	1–3 Mbps per call can exceed Voice's 25% threshold at 20 Mbps; matches WebRTC guidance (RFC 8837: audio EF, video AF4x)
Streaming	AF31 (26)	Video	Buffered; needs throughput, not lowest delay
Interactive-Web, Unknown	CS0 (0)	Best Effort	Never boosted on a guess
Bulk	CS1 (8)	Bulk	Downloads, updates, backups

Tin thresholds are soft: a tin over its threshold drops to lower priority for the excess, it is not capped. Hard per-class floors exist only in the HTB variant.

One mark, both directions FIX

1. **Write:** when the classifier (or an intent) decides a flow's class, the controller stores it in the flow's conntrack mark: DSCP in bits 26–31 (mask `0xfc000000`) plus a "valid" flag bit `0x01000000`. Example for EF: (46 << 26) | 0x01000000 = `0xb9000000`. Written with `conntrack -U … --mark` (or libnetfilter_conntrack).
2. **Upload:** a `ctinfo` action on the LAN interface ingress restores DSCP from the mark before routing. CAKE on WAN egress then picks the tin. `wash` clears our DSCP after queuing so it never leaks to the ISP.
3. **Download:** on WAN ingress, first **clear any DSCP set by the Internet** (`pedit … dsfield set 0 retain 0xfc`; IPv6 `traffic_class`), then restore ours with `ctinfo`, then redirect to IFB, where CAKE picks the tin. A remote sender marking EF therefore gains nothing.

```
# Upload: CAKE on WAN egress
tc qdisc replace dev wan0 root cake bandwidth 18mbit diffserv4 dual-srchost nat wash ack-filter
ethernet
tc qdisc add dev lan0 handle ffff: ingress
tc filter add dev lan0 parent ffff: protocol all prio 10 matchall \
   action ctinfo dscp 0xfc000000 0x01000000

# Download: clear untrusted DSCP, restore ours, redirect to IFB, shape there
ip link add ifb0 type ifb && ip link set ifb0 up
tc qdisc replace dev ifb0 root cake bandwidth 90mbit diffserv4 dual-dsthost nat ingress ethernet
tc qdisc add dev wan0 handle ffff: ingress
tc filter add dev wan0 parent ffff: protocol ip prio 10 matchall \
   action pedit ex munge ip dsfield set 0 retain 0xfc pipe \
   action ctinfo dscp 0xfc000000 0x01000000 pipe \
   action mirred egress redirect dev ifb0
tc filter add dev wan0 parent ffff: protocol ipv6 prio 11 matchall \
   action pedit ex munge ip6 traffic_class set 0 retain 0xfc pipe \
   action ctinfo dscp 0xfc000000 0x01000000 pipe \
   action mirred egress redirect dev ifb0

# Fast loop after a capacity drop (no trial, no hold time)
tc qdisc change dev ifb0 root cake bandwidth 18mbit
```

Illustrative values; exact rates come from the controller. `ethernet` is a placeholder overhead keyword.

Link realism NEW

- **Overhead compensation:** set CAKE's overhead keyword for the access type (e.g. `ethernet`, `docsis`, `pppoe-ptm`, or explicit `overhead N mpu M` for PPPoE-over-fibre). Without it the shaper under-counts bytes and the queue moves to the modem.
- **ACK filtering:** `ack-filter` on the upload of asymmetric links, so download ACKs don't crowd out call upload.
- **GSO and offloads:** CAKE splits GSO super-packets by default (`split-gso`). In the HTB variant and in the lab, disable GRO/GSO/TSO on veth links (`ethtool -K`) so the emulation doesn't hide queueing.
- **ECN:** CAKE uses ECN when available; run with ECN on and off and report both.

HTB variant (hard floors)

One HTB tree per direction with a class per category: `rate` = guaranteed floor, `ceil` = maximum, fq_codel leaves. The validator keeps the sum of floors below the shaped rate and scales them down when capacity drops. This variant gives the guarantees; the CAKE layout gives priority plus per-host fairness.

Known limits of a router-side shaper

- Download shaping controls only the router's queue; ISP buffers are out of reach, which is why the shaper runs slightly below capacity.
- DSCP is used only inside the router; marks on the Internet are not relied on.
- **VPN** NEW : traffic inside one VPN tunnel is a single flow, so Focus Mode cannot separate a call inside it. Per-host fairness still applies; this is reported as a limitation.

Week-1 verification checklist (kernel behaviour we test, not assume)

- The mark survives NAT, and `ctinfo` restores DSCP on LAN ingress and WAN ingress for IPv4 and IPv6.
- Inbound EF is cleared before restoration (abuse test S12).
- CAKE tin counters (`tc -s qdisc`) show each class in the intended tin.
- A rate change through `tc qdisc change` takes effect without dropping existing flows.

7. Where AI is used, and how it must earn its place

AI part	Technique	Deterministic twin	Must improve
Traffic understanding	LightGBM + Mondrian conformal prediction	Port/hostname rules	Wrong-boost rate ≤ α, higher macro-F1, correct QUIC/unknown handling, time-to-classify
Experience Twin	LightGBM quantile regression on offline lab data	Safety gate without pre-screen	Fewer harmful trials and harm-seconds at equal or better p95 delay
Zero-click Focus	Conformal singleton call detection + budgeted auto-intent	Manual intent only	Shorter time-to-protection, fewer harm-seconds at call start
Policy tuner (stretch)	Contextual bandit (LinUCB) inside the screened menu	Rule controller	Lower p95 delay or better bulk progress at equal fairness

If an AI part does not beat its twin in repeated A/B runs, it is switched off and the result is reported. That rule itself is part of the pitch.

8. Data and ethics

- Own captures: recorded only on the team's own devices and accounts, with consent. Only metadata features are kept; raw pcaps are not published.
- Public datasets: used only under their licences, listed in the licence table.
- Synthetic traffic: used for load and stress only, never as classifier ground truth.

9. Evaluation plan

Lab: Linux network namespaces with real kernel networking: *home* (emulated devices), *router* (PulseQoS + tc), *isp* (token bucket + netem for rate, delay and loss). Every report is labelled emulation.

Traffic:

- Call audio: about 50 packets/s, small packets.
- FIX Call video: 1–3 Mbps with frame bursts, both directions. Audio-only would hide the Voice-threshold issue.
- Gaming: small, high-rate UDP.
- Adaptive video: TCP bursts with idle gaps.
- Bulk: `iperf3` with multiple streams.
- Measurement: Flent RRUL, `irtt`, and RPM via `goresponsiveness` NEW .

Baselines

#	System	Why
1	FIFO, no QoS	Raw bufferbloat
2	Static CAKE (diffserv4, fixed rate)	Strong standard AQM
3	CAKE `autorate-ingress` NEW	CAKE's own built-in capacity estimation (tc-cake(8))
4	CAKE + cake-autorate	Existing delay-based adaptive rate

5	Static rules + CAKE (port/hostname)	Existing app-aware QoS
6	PulseQoS deterministic core	Our system without AI
7	PulseQoS + AI layer (conformal, Twin, zero-click)	Measures what the AI adds
8	7 + bandit tuner (stretch)	Measures online learning

Scenarios

#	Scenario	Primary metrics
S1	ISO download starts during a call	Call p95/p99 delay, jitter, loss; bulk goodput
S2	WAN 100 → 20 Mbps	Settle time, delay-spike size, oscillation
S3	Three TVs + gaming	Gaming p95, video sufficiency, Jain
S4	Focus Mode under load	Before/after, floors held, expiry on time
S5	Conflicting intents	Arbitration correctness, fairness
S6	Bulk mislabelled as interactive	Wrong-boost duration, recovery after correction
S7	Unknown QUIC/VPN flow	Abstention, no aggressive boost
S8	Controller crash	Time to safe state
S9	IPv4 vs IPv6	Parity
S10 NEW	Call starts while a download is already running	Time-to-classify, time-to-protection, first-minute harm-seconds
S11 NEW	Two video calls + gaming at 20 Mbps	Voice/Video tin behaviour, call p95
S12 NEW	External flood pre-marked EF	Flood stays in Best Effort; call unaffected
S13 NEW	Upload-only congestion (cloud backup)	Correct direction cut (OWD attribution)
S14 NEW	Bad policy forced while capacity also drops	Inconclusive-trial handling, no wrong blame

Metrics

p50/p95/p99 delay, jitter and loss for interactive flows (end to end, from probes and generators); RPM; video throughput sufficiency; bulk goodput and completion time; Jain's index across hosts with similar demand; demand-aware shortfall; starvation events; capacity-estimate error; macro-F1, wrong-boost rate vs α, abstention, time-to-classify; harm-seconds; settle time; changes per minute; rollbacks; CPU and memory.

Protocol

At least 10 runs per scenario × system, with randomised order, fixed seeds and paired comparisons. We report medians with bootstrap 95% confidence intervals *of the difference* and effect sizes. All raw data, netem settings and scripts are committed. Where PulseQoS only ties a baseline, we report it.

10. Pilot result (measured 27 Oct 2026)

System (20 Mbit bottleneck, 4-stream download)	Call-probe p95	p99	Jitter	Download goodput
FIFO (1000-packet queue)	**138 ms** (128–152)	153 ms	8.5 ms	18.7 Mbps
HTB class layout (interactive first, 10% bulk floor)	**0.07 ms**	0.10 ms	0.01 ms	18.7 Mbps

5 paired runs, medians. Linux namespaces in a cloud container where netem, CAKE and fq_codel modules were unavailable, so: no WAN delay, ICMP probes as the call proxy, ideal classification. This shows the problem and the mechanism, not PulseQoS's advantage over CAKE, which is the Round 2 question. Scripts and raw data: `pulseqos/pilot/`.

11. Demo (about 4 minutes) and backup

1. **Household at 100 Mbps:** all classes healthy.
2. **Download starts:** FIFO replay shows the call spiking; PulseQoS holds it. The "Why" panel explains the decision.
3. **ISP drops to 20 Mbps:** the fast loop cuts within a second, shown next to CAKE autorate on the same replay.
4. **A call starts on another laptop:** Zero-click Focus protects it ("Undo" visible). The Twin's predicted delay is shown beside the measured one.
5. **Break it:** force a bad policy; the Twin rejects it. Force it past the Twin; the fast-abort rolls it back. Kill the controller; the watchdog restores the safe state.
6. **One command** produces the comparison report with confidence intervals.

NEW A recorded backup video of the same run is kept, in case the live environment (root access, kernel modules) is unavailable on demo day.

12. Tough judge questions

Where is the AI, really? In three places, each with a number it must beat: the conformal classifier (wrong-boost rate bound), the Experience Twin (harm-seconds avoided before users feel them) and Zero-click Focus (time-to-protection). Each is compared with a deterministic twin and switched off if it loses.

Why not an LLM? The case study says AI must improve a measurable outcome, not add a conversational interface. A natural-language intent box could be added on top later; it is not where the value is.

CAKE already works. Why PulseQoS? We run on CAKE and benchmark against CAKE, its autorate and cake-autorate. CAKE has no notion of user intent, doesn't predict or verify its own changes, and doesn't bound classification mistakes.

Can the classifier be fooled or wrong? Yes. Uncertain flows abstain to Unknown, the wrong-boost rate is bounded and monitored, users can correct labels, and external DSCP marks are ignored.

Is the bounded trial an experiment on users? Yes, and that's why the Twin screens first, the trial is short, the fast-abort reacts in about two seconds, and the watchdog guarantees a safe fallback.

What changes for a product? Router hardware, hardware flow offload (which can bypass shaping), CPU for gigabit shaping, Wi-Fi airtime fairness, a security review of the intent API, and certification and scale testing.

13. Risks and limitations

Risk	Mitigation
Classifier weak on new apps, QUIC, VPN	Conformal abstention, drift monitor, deterministic fallback, user correction
Twin wrong outside the lab (sim-to-real)	It can only remove actions; live trial + fast-abort still decide; prediction error reported
CAKE alone already very good	Honest comparison; claims focused on intents, capacity changes and safety
Policy flapping	Slow-loop hysteresis, hold time, change budget; fast loop only cuts quickly
Bulk starvation	HTB floors; CAKE minimum-progress monitoring; Fairness Budget
Fast-abort misfires on noise	Two consecutive windows; margins tuned on baseline runs
Wrong-direction rate cut	One-way delay; load-based attribution fallback
DSCP abuse from the Internet	Clear then restore on ingress; abuse test S12
Intent API abuse	LAN-only, per-user tokens, rate limits, budgets, audit
Overstated QoE claims	Network-level proxies only, clearly labelled; no MOS claim
Emulation ≠ real Wi-Fi homes	Labelled; optional OpenWrt run as a stretch goal
Router CPU	Small models, batched telemetry, CPU reported as a metric
Demo environment fails	Recorded backup video

14. Tech stack and Round 2 plan

Component	Technology
Emulation	Linux network namespaces, netem, token-bucket shaping; Ubuntu 24.04 VM with root
Enforcement	tc: CAKE, IFB, HTB + fq_codel; actions ctinfo, pedit, mirred; conntrack-tools
Telemetry	conntrack, tc -s, libpcap/eBPF sampling, tsping/irtt probes
AI	Python, LightGBM, scikit-learn; conformal prediction (MAPIE or own implementation)
Controller and API	Python, FastAPI
Benchmarking	iperf3, Flent (RRUL), irtt, goresponsiveness
Dashboard	React + Vite, Recharts
Storage and tests	SQLite + JSON/CSV artifacts; pytest; scripted namespace resets

Week	Goal
1	Namespace lab, traffic generators (incl. call video), baselines 1–5, verification checklist (section 6)
2	Fast loop with OWD, slow-loop controller, intents, validator, rollback, watchdog
3	Conformal classifier on real captures, Experience Twin (offline data + model), Zero-click Focus, dashboard and audit

4	Full scenarios S1–S14 with repeated runs, report generation, deck, demo and backup video

Confirm the Round 2 dates on the portal before quoting them. Scope order: core (deterministic) must be reliable before the AI layer; the bandit tuner and OpenWrt run are stretch goals.

15. Changes from v1

#	Change	Type
1	Two-speed control; rate cuts exempt from trial and hold time; inconclusive-trial rule	Correctness
2	Call video moved to the Video tin (25% Voice threshold; RFC 8837)	Correctness
3	Untrusted inbound DSCP cleared before restoring our mark	Security
4	Specified how the conntrack mark is written and restored in both directions	Correctness
5	One-way delay for per-direction capacity estimation	Correctness
6	Overhead compensation, ACK filter, GSO/offload handling, ECN	Correctness
7	CAKE autorate-ingress added as a baseline	Evaluation
8	Classifier trained on real captures; time-to-classify; RPM; paired bootstrap statistics	Evaluation
9	Five new scenarios (S10–S14)	Evaluation
10	Intent API security; household privacy; VPN limitation	Security / privacy
11	AI layer: conformal classifier, Experience Twin, Zero-click Focus, each with a deterministic twin	Differentiation
12	Six-layer architecture view; expected-output checklist; backup demo video	Submission
13	Measured pilot added	Evidence

16. Sources

- Tata Elxsi TELIPORT S4, Case Study 3 PDF (constraints, expected output, evaluation criteria)
- tc-cake(8), tc-ctinfo(8), tc-pedit(8): iproute2 manual pages (tin thresholds, wash, nat, dual-host modes, overhead keywords, split-gso, autorate-ingress, ctinfo DSCP restore, pedit dsfield)
- RFC 8837: DSCP Packet Markings for WebRTC QoS
- IETF IPPM draft: Responsiveness under Working Conditions (RPM); goresponsiveness client
- cake-autorate and tsping (OpenWrt community); qosify; LibreQoS
- Vovk, Gammerman, Shafer: Algorithmic Learning in a Random World (conformal prediction); Angelopoulos & Bates, A Gentle Introduction to Conformal Prediction
- ISCX VPN-nonVPN 2016 (UNB CIC); CESNET-QUIC22: check licences before use